# ASR STUDIO: AN END-TO-END AUTOMATIC SPEECH RECOGNITION STUDIO USING OPENAI WHISPER, FASTAPI AND REACT (NODE.JS)

**On-Device Faster-Whisper Inference, Voice Activity Detection, Multi-Format Subtitle Export and Batch Processing**

**A PROJECT REPORT**

*Submitted by*

**KARTHIG RAJAN [Reg No: RA2311003050204]**
**SACHIN KUPPUSWAMY [Reg No: RA2311003050263]**
**NANDAKISHORE K S [Reg No: RA2311003050216]**

*Under the guidance of*

**Dr. R. Baskar**
(Assistant Professor, Department of Computer Science & Engineering)

in partial fulfilment of the requirements for the degree of
**BACHELOR OF TECHNOLOGY**
in
**COMPUTER SCIENCE AND ENGINEERING**




**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**
**FACULTY OF ENGINEERING AND TECHNOLOGY**
**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY**
**TIRUCHIRAPPALLI – 621105**
**OCTOBER 2026**

# TABLE OF CONTENTS

**AI / LLM Declaration** **2**

**Project Details at a Glance** **2**

**1 Executive Summary** **3**

**2 Introduction & Background** **3**

**3 Problem Statement & Motivation** **3**

**4 Project Objectives & Scope** **4**

**5 Key Features & Specifications** **4**

**6 System Architecture & High-Level Design** **6**

**7 Technology Stack & Dependencies** **8**

**8 Data Model & Configuration Schema** **9**

**9 REST API Resource Endpoints** **10**

**10 Module Breakdown & Implementation Details** **11**

**11 User Interface Walkthrough** **14**

**12 Command-Line Interface** **17**

**13 Installation & Deployment Guide** **18**

**14 Testing & Quality Assurance** **19**

**15 Performance Metrics & Benchmarks** **20**

**16 Security & Privacy Considerations** **20**

**17 Comparison with Commercial Solutions** **21**

**18 Limitations** **21**

**19 Future Enhancements** **21**

**20 Conclusion & References** **22**

## AI / LLM DECLARATION

I/We used Gemini / Claude (Gemini 3.7 / Claude 3.7) to assist in architecting the audio preprocessing workflow, structuring OpenAPI endpoint documentation, optimizing PyTest unit test coverage, and designing CSS design tokens for the Emerald Matrix UI theme.

The extent of AI/LLM usage is approximately 15–20%, limited to architectural suggestions, documentation formatting, and visual layout references.

All core backend implementation logic, CTranslate2 Faster-Whisper model inference orchestration, audio signal resampling buffers, subtitle exporter serialization, CLI parser development, React component engineering, and end-to-end integration were done manually by the team.

## PROJECT DETAILS AT A GLANCE

| | |
|---|---|
| **Project Title** | Automatic Speech Recognition (ASR) Studio |
| **Team Members** | Karthig Rajan (RA2311003050204), Sachin Kuppuswamy (RA2311003050263), Nandakishore K S (RA2311003050216) |
| **Guide** | Dr. R. Baskar, Assistant Professor, Dept. of CSE |
| **Course** | Modern Application Development |
| **Frontend** | React 19 + Vite (Node.js), Lucide icons, HTML5 Web Audio API |
| **Backend** | Python 3.10+, FastAPI, Uvicorn |
| **ASR Core** | Faster-Whisper (CTranslate2, int8) with SpeechRecognition fallback |
| **Repository** | https://github.com/Rajan3103/asr-studio |

## 1. EXECUTIVE SUMMARY

ASR Studio is an end-to-end, browser-based Automatic Speech Recognition (ASR) platform that converts spoken audio into accurate, timestamped text. Users can either record their voice live through the browser microphone or upload pre-recorded audio files in WAV, MP3, FLAC, M4A, AAC and OGG formats. The audio is sent to a FastAPI microservice, where it is processed by Faster-Whisper, a CTranslate2-optimised re-implementation of OpenAI's Whisper transformer model, running with 8-bit integer (int8) quantisation on a standard CPU.

The system returns the full transcript, the detected language with its probability, per-segment start/end timestamps, a confidence score for every segment, and optional word-level timestamps. In addition to transcription, the same model can translate speech from more than 90 languages directly into English. Results can be copied in one click or downloaded as plain text (.txt), SubRip subtitles (.srt), WebVTT captions (.vtt), structured JSON (.json) or spreadsheet-ready CSV (.csv).

Around this core, the studio provides a synthetic-audio latency benchmark, a multi-file batch processing queue with live progress, an interactive REST API reference, a command-line interface (CLI) for scripted and directory-level transcription, and five switchable visual themes. The primary recognition path runs entirely on the user's own machine, so recorded speech is never sent to a paid cloud API, and the whole code base is covered by an automated suite of 20 PyTest unit and integration tests.

## 2. INTRODUCTION & BACKGROUND

Speech is the most natural way humans communicate, yet most digital systems still depend on typed text for search, documentation and accessibility. Automatic Speech Recognition bridges this gap by converting an acoustic signal into a sequence of words. Classical ASR systems were built from separately trained components: an acoustic model (often Gaussian Mixture Models or Hidden Markov Models), a pronunciation lexicon and an n-gram language model. These pipelines needed large amounts of hand-aligned data for every language and were fragile in the presence of accents, background noise and domain-specific vocabulary.

The arrival of deep learning, and in particular the Transformer architecture (Vaswani et al., 2017), changed this landscape. In 2022 OpenAI released Whisper, an encoder–decoder Transformer trained on 680,000 hours of weakly supervised multilingual audio. Whisper performs transcription, language identification and speech translation with a single network and is robust to noise and accents. However, the reference PyTorch implementation is heavy for ordinary laptops. Faster-Whisper re-implements Whisper on top of the CTranslate2 inference engine, which supports int8 quantisation and optimised CPU kernels, making on-device transcription practical without a GPU.

Commercial services such as Google Cloud Speech-to-Text, AWS Transcribe and Otter.ai offer high accuracy but require uploading audio to third-party servers and charge per minute of audio. For students, researchers and institutions handling lectures, interviews or confidential meetings, this raises privacy and cost concerns. ASR Studio was built to show that a modern web stack (React + FastAPI) combined with an optimised open-source model can deliver a polished, privacy-respecting transcription and captioning tool that runs on everyday hardware.

## 3. PROBLEM STATEMENT & MOTIVATION

**Problem Statement:** To design and develop a high-performance, browser-based Automatic Speech Recognition (ASR) Studio that allows users to capture live voice via browser Web Audio APIs or upload multi-format audio files (WAV, MP3, FLAC, M4A, AAC, OGG) and receive accurate speech-to-text transcriptions, translation to English, and millisecond-level timed subtitle segments, with the primary inference path running fully on-device without paid cloud API dependencies.

Current deficiencies in everyday transcription and captioning workflows include:

- **Cloud Privacy Risks:** Popular transcription services upload raw audio to external servers. Lectures, interviews, medical notes or internal meetings may contain sensitive information that should not leave the organisation.
- **Recurring Per-Minute Costs:** Cloud ASR APIs are billed per minute of audio. Departments that transcribe many hours of lectures every week face continuous operational expense.
- **Unstructured Output:** Many simple dictation tools return a single block of text without timestamps, which is useless for creating subtitles or locating a moment in a long recording.
- **Fragmented Tooling:** Recording, format conversion, transcription and subtitle generation are usually done in separate applications, forcing users to move files between tools.
- **Hardware Barrier:** The original Whisper implementation runs slowly on CPU-only laptops, which are the most common hardware available to students.
- **No Single-Point Failure Handling:** If a local model fails to load (for example, due to insufficient memory), most tools simply crash instead of degrading gracefully.

## 4. PROJECT OBJECTIVES & SCOPE

ASR Studio was engineered to fulfil the following objectives:

- **On-Device Inference:** Run Whisper models locally through Faster-Whisper with int8 CPU quantisation so that the primary path needs no paid cloud API.
- **Flexible Audio Ingestion:** Accept live microphone recordings and six common audio formats, and normalise them into 16 kHz mono float32 signals.
- **Rich, Structured Results:** Return transcript text, detected language, language probability, audio duration, processing time, timed segments, confidence and word timestamps.
- **Multi-Format Export:** Generate TXT, SRT, VTT, JSON and CSV outputs that can be used directly in video editors, media players and spreadsheets.
- **Configurable Inference:** Let the user choose model tier (tiny to large-v3), task (transcribe/translate), source language, beam size, VAD filtering and word timestamps.
- **Resilience:** Fall back automatically to a secondary SpeechRecognition engine if the Whisper model cannot run.
- **Multiple Interfaces:** Expose the same engine through a React web studio, a documented REST API and a command-line tool.
- **Quality Assurance:** Validate every module with automated PyTest unit and integration tests.

**Scope.** The project covers single-user transcription of recorded or uploaded audio, translation into English, subtitle generation and batch processing. Real-time word-by-word streaming, speaker diarisation (“who spoke when”) and user accounts/persistent storage are outside the current scope and are listed as future enhancements.

## 5. KEY FEATURES & SPECIFICATIONS

### 5.1 Audio Capture & Ingestion

- Drag-and-drop or click-to-browse upload supporting WAV, MP3, FLAC, M4A, AAC and OGG.

- Live microphone recording through `navigator.mediaDevices.getUserMedia` and the `MediaRecorder` API, with a running recording timer.
- Real-time frequency-bar visualiser drawn on an HTML5 canvas using a Web Audio `AnalyserNode` (FFT size 128).
- Automatic audio signal analysis on selection: duration, sample rate, RMS energy, peak amplitude and silence detection.

### 5.2 Speech Recognition Engine

- Faster-Whisper (CTranslate2) with int8 quantisation running on CPU by default, CUDA optional through the CLI.
- Five model tiers: `tiny`, `base` (default), `small`, `medium`, `large-v3`.
- Transcribe mode (same language) and Translate mode (any supported language → English).
- Automatic language detection or manual selection among 15 commonly used languages in the UI (Whisper itself supports 90+).
- Silero Voice Activity Detection (VAD) filter to skip silent and non-speech regions.
- Configurable beam search width (1–10) and word-level timestamps.
- Model instances are cached per tier, so the model is loaded from disk only once per server lifetime.
- Automatic fallback to the SpeechRecognition library if Faster-Whisper raises an error.

### 5.3 Results Viewer & Export Hub

- Full transcript panel with one-click copy to clipboard.
- Metadata cards: detected language, language probability, audio duration, processing time and engine used.
- Timed segments inspector listing start/end time, text and confidence for every segment.
- One-click download in five formats: TXT, SRT, VTT, JSON and CSV.

### 5.4 Productivity & Developer Tools

- **Synthetic Benchmark:** generates a 1–10 second harmonic test signal and measures end-to-end pipeline latency.
- **Batch Queue:** upload several files and process them sequentially with a live progress bar.
- **REST API & CLI Docs tab:** in-app reference with ready-to-copy `curl` and CLI commands; interactive Swagger UI at `/docs`.
- **Theme Engine:** five themes – Emerald Matrix / Dark Velvet, Cyber Obsidian, Deep Nebula, Tokyo Sunset and Nordic Frost.

## 6. SYSTEM ARCHITECTURE & HIGH-LEVEL DESIGN

ASR Studio follows a decoupled client–server architecture. The React single-page application handles all user interaction and talks to the Python backend exclusively through HTTP requests with multipart form-data payloads. The backend is organised as a pipeline of small, single-purpose modules that can also be reused independently by the command-line tool.

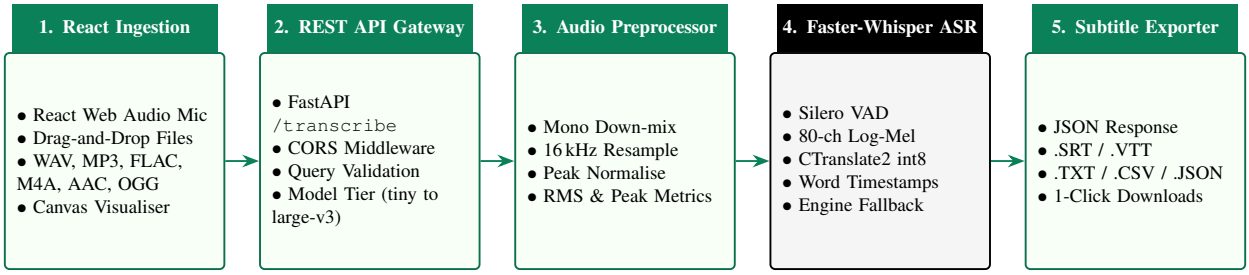



*Figure 1: End-to-end system architecture and audio pipeline of ASR Studio*

| Architectural Tier | Components & Technologies | Core Responsibilities |
|---|---|---|
| **Tier 1: Presentation & Audio Ingestion** | React 19, Vite, Lucide icons, Web Audio API, MediaRecorder, Canvas | File upload, live recording, visualiser, model configuration, results viewer, theme switching |
| **Tier 2: REST API Gateway** | FastAPI, Uvicorn, python-multipart, CORS middleware | Endpoint routing, enum-based query validation, temporary file handling, error mapping to HTTP 400/500 |
| **Tier 3: Signal Processing** | SoundFile, SciPy `signal.resample`, NumPy, `wave` | Decoding, mono conversion, 16 kHz resampling, normalisation, metrics, synthetic signal generation |
| **Tier 4: ASR Inference** | Faster-Whisper, CTranslate2, Silero VAD, SpeechRecognition | Model caching, transcription, translation, language detection, timestamps, confidence, fallback |
| **Tier 5: Serialisation & Export** | `json`, `csv`, custom timecode formatter | TXT/SRT/VTT/JSON/CSV generation and file streaming with `Content-Disposition` headers |

### 6.1 Request Sequence for a Transcription

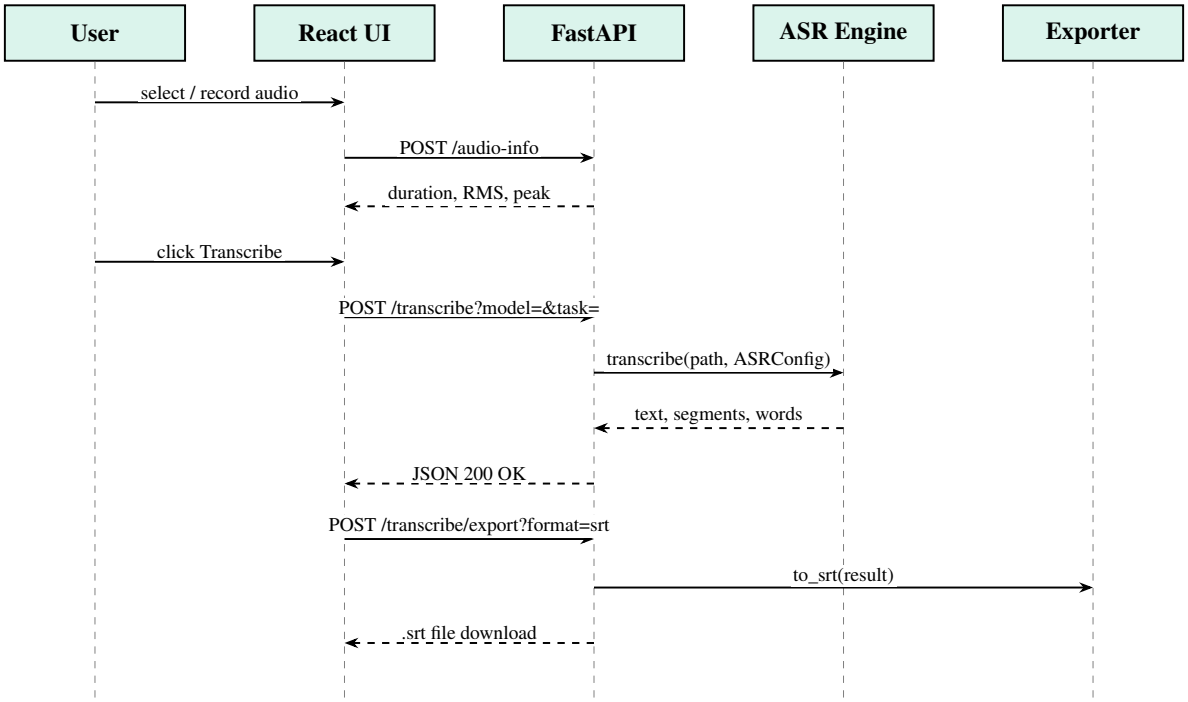



*Figure 2: Sequence of HTTP calls for analysing, transcribing and exporting one audio file*

### 6.2 Engine Fallback Logic

The `UnifiedASREngine` acts as a facade over two concrete engines that both implement the abstract `BaseASREngine.transcribe()` interface. This strategy pattern lets the API layer remain unaware of which engine actually produced the result.

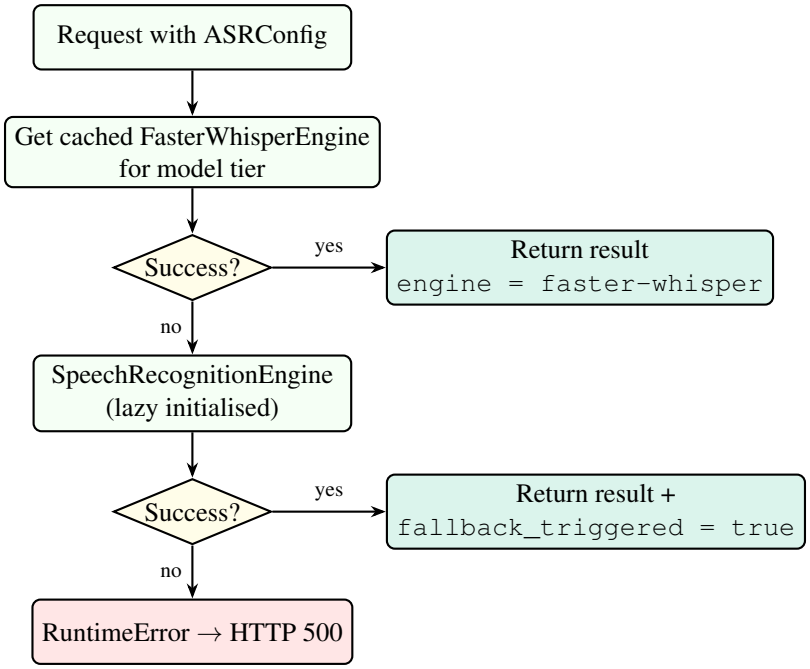



*Figure 3: Primary/fallback decision flow inside `UnifiedASREngine`*

## 7. TECHNOLOGY STACK & DEPENDENCIES

Comprehensive summary of the software packages, libraries and frameworks used (minimum versions are taken from `requirements.txt` and `frontend/package.json`):

| Subsystem | Technology / Library | Version | Role & Engineering Purpose |
|---|---|---|---|
| Backend Runtime | Python | 3.10+ | Core language for API, engine, CLI and tests |
| Web Framework | FastAPI | ≥0.100 | Async REST endpoints, validation, OpenAPI/Swagger docs |
| ASGI Server | Uvicorn | ≥0.22 | High-performance server hosting the FastAPI app |
| Multipart Parser | python-multipart | ≥0.0.6 | Parsing uploaded audio files in form-data requests |
| ASR Engine | Faster-Whisper | ≥1.0 | CTranslate2 Whisper inference with int8 quantisation |
| Tensor Runtime | PyTorch | ≥2.0 | Deep learning runtime used by the VAD/ASR stack |
| Fallback ASR | SpeechRecognition | ≥3.10 | Secondary engine used when Whisper is unavailable |
| Audio I/O | SoundFile | ≥0.12 | Decoding audio files into float32 arrays |
| Signal Processing | SciPy / NumPy | ≥1.10 / 1.24 | Resampling, normalisation, RMS and peak metrics |
| Frontend Library | React | 19.2 | Component-based single-page user interface |
| Build Tool | Vite | 8.x | Dev server (port 3000), hot reload, production build |
| Icons | lucide-react | 1.x | Vector icons for controls and cards |
| Audio Visualiser | Web Audio API / Canvas | W3C | Live frequency bars via `AnalyserNode` |
| Recording | MediaRecorder API | W3C | Capturing microphone audio in the browser |
| Linting | oxlint | 1.x | Fast JavaScript/JSX static analysis |
| Testing | PyTest / HTTPX | ≥7.4 / 0.24 | Unit and API integration tests via `TestClient` |

### 7.1 Whisper Model Tiers

| Model | Parameters | Typical Use |
|---|---|---|
| tiny | ∼39 M | Fastest; quick drafts and low-end hardware |
| base (default) | ∼74 M | Best balance of speed and accuracy on CPU |
| small | ∼244 M | Better accuracy for accented or noisy speech |
| medium | ∼769 M | High accuracy; slower on CPU |
| large-v3 | ∼1550 M | Highest accuracy; GPU recommended |

## 8. DATA MODEL & CONFIGURATION SCHEMA

ASR Studio is a stateless service: it does not keep a database. Every request is self-contained, uploaded audio is written to a temporary file, processed, and deleted in a `finally` block. The data model therefore consists of the configuration object passed to the engine and the structured JSON result returned to the client.

### 8.1 ASRConfig Dataclass (`config.py`)

| Field | Type | Default | Description |
|---|---|---|---|
| model_size | ModelSize enum | base | Whisper tier: tiny, base, small, medium, large-v3 |
| device | str | cpu | Inference device: cpu or cuda |
| compute_type | ComputeType enum | int8 | float32, float16, int8, int8_float16 |
| task | TaskType enum | transcribe | transcribe or translate (to English) |
| language | Optional[str] | None | ISO code, None = auto-detect |
| beam_size | int | 5 | Beam search width (API allows 1–10) |
| vad_filter | bool | True | Enable Silero voice activity detection |
| word_timestamps | bool | True | Return per-word start/end/probability |
| sample_rate | int | 16000 | Target sampling rate in Hz |
| temperature | float | 0.0 | Decoding temperature (0 = greedy/beam) |
| initial_prompt | Optional[str] | None | Optional context prompt for decoding |

### 8.2 Transcription Result Schema

| Field | Type | Description |
|---|---|---|
| filename | string | Original uploaded file name |
| text | string | Full transcript (segments joined by spaces) |
| language | string | Detected or forced ISO language code |
| language_probability | float | Confidence of language detection (0–1) |
| duration_seconds | float | Length of the audio in seconds |
| processing_time_seconds | float | Wall-clock inference time |
| engine | string | `faster-whisper` or `speech-recognition-fallback` |
| model_size | string | Model tier used |
| segments[] | array | id, start, end, text, confidence, words[] |
| segments[].words[] | array | word, start, end, probability |
| fallback_triggered | bool | Present only when the fallback engine was used |
| fallback_reason | string | Error message from the primary engine |

Segment confidence is derived from Whisper's average log-probability: confidence $= e^{\overline{\log p}}$, rounded to three decimals.

## 9. REST API RESOURCE ENDPOINTS

| Endpoint | Method | Description |
|---|---|---|
| `/` | GET | Service name, author, version, status and documentation links |
| `/health` | GET | Health check returning `{"status": "healthy"}` |
| `/models` | GET | Available Whisper tiers, supported languages and default model |
| `/audio-info` | POST | Audio metrics: samples, sample rate, duration, RMS, peak, is_silent |
| `/synthesize-sample` | POST | Generates a synthetic 1–10 s 16 kHz PCM WAV for benchmarking |
| `/transcribe` | POST | Transcribes/translates audio and returns the JSON result |
| `/transcribe/export` | POST | Transcribes and streams a downloadable .srt/.vtt/.txt/.json/.csv file |
| `/docs`, `/redoc` | GET | Interactive Swagger UI and ReDoc OpenAPI documentation |

### 9.1 Query Parameters of `/transcribe`

| Parameter | Type | Default | Validation / Meaning |
|---|---|---|---|
| model | enum | base | tiny \| base \| small \| medium \| large-v3 |
| task | enum | transcribe | transcribe \| translate |
| language | string | auto | ISO code or omitted for detection |
| beam_size | int | 5 | $1 \leq$ beam_size $\leq 10$ |
| vad_filter | bool | true | Enable Silero VAD |
| word_timestamps | bool | true | Include word-level timing |

### 9.2 Request / Response Example

```
// Request: POST /transcribe?model=base&task=transcribe
// Content-Type: multipart/form-data   [file=@speech_sample.wav]

// Response (200 OK)
{
  "filename": "speech_sample.wav",
  "text": "Welcome to the Automatic Speech Recognition Studio demonstration.",
  "language": "en",
  "language_probability": 0.985,
  "duration_seconds": 3.42,
  "processing_time_seconds": 0.284,
  "engine": "faster-whisper",
  "model_size": "base",
  "segments": [
    { "id": 0, "start": 0.0, "end": 3.42,
      "text": "Welcome to the Automatic Speech Recognition Studio demonstration.",
      "confidence": 0.98,
      "words": [
        { "word": "Welcome", "start": 0.0,  "end": 0.42, "probability": 0.99 },
        { "word": "to",      "start": 0.44, "end": 0.58, "probability": 0.99 } ] } ]
}
```

*Values are illustrative; the field names match the backend implementation exactly.*

Validation failures (e.g. an unknown model name) are rejected by FastAPI with HTTP 422; unreadable audio returns HTTP 400 from `/audio-info`, and inference errors return HTTP 500 with a descriptive `detail` message.

## 10. MODULE BREAKDOWN & IMPLEMENTATION DETAILS

The code base is organised into decoupled modules:

- **asr_tool/api.py:** FastAPI application. Registers CORS middleware, creates one shared `UnifiedASREngine`, and defines the seven endpoints. Uploaded files are written to a `NamedTemporaryFile` with the original extension and always removed in a `finally` block.
- **asr_tool/config.py:** Enumerations (`ModelSize`, `ComputeType`, `TaskType`, `OutputFormat`), the `SUPPORTED_LANGUAGES` map and the `ASRConfig` dataclass with a `to_dict()` serialiser.
- **asr_tool/audio_processor.py:** `AudioProcessor` with `load_audio` (decode, mono down-mix, resample, normalise), `get_audio_info` (metrics), `create_synthetic_speech_wav` and `save_wav`.
- **asr_tool/engine.py:** Abstract `BaseASREngine`, `FasterWhisperEngine` (lazy model loading,

segment/word extraction, confidence), `SpeechRecognitionEngine` (fallback) and `UnifiedASREngine` (per-tier cache + fallback).

- **asr_tool/exporters.py:** `TranscriptExporter` with SRT/VTT timecode formatters and `to_txt/to_srt/to_v`
- **asr_tool/cli.py:** `argparse` CLI with `transcribe`, `info` and `generate-sample` sub-commands, including recursive directory batch mode.
- **frontend/src/App.jsx:** The complete React studio (about 1,000 lines): state management with hooks, recording and visualiser, API calls, results viewer, benchmark, batch queue, API docs and theme switcher.
- **frontend/src/index.css:** Design system with CSS custom properties for the five themes, glassmorphism cards and responsive layout.
- **frontend/vite.config.js:** Dev server on port 3000 with a `/api` proxy to `127.0.0.1:8000`.
- **tests/:** Five PyTest modules covering API, audio processor, CLI, engine and exporters.
- **generate_sample_audio.py:** Helper script that creates sample audio for manual testing.

### 10.1 Audio Preprocessing Algorithm

```
audio, sr = sf.read(path, dtype="float32")          # 1. decode
if audio.ndim > 1:                                   # 2. stereo -> mono
    audio = np.mean(audio, axis=1)
if sr != 16000:                                      # 3. resample to 16 kHz
    n = int(len(audio) * 16000 / sr)
    audio = signal.resample(audio, n).astype(np.float32)
peak = np.max(np.abs(audio))                         # 4. peak normalise
if peak > 0:
    audio = audio / peak
```

The signal metrics are computed as

$$\text{RMS} = \sqrt{\frac{1}{N}\sum_{n=1}^{N} x[n]^2}, \qquad \text{Peak} = \max_n |x[n]|, \qquad \text{is_silent} = (\text{RMS} < 10^{-4}).$$

### 10.2 Whisper Inference Pipeline

Inside Faster-Whisper, the 16 kHz waveform is split into 30-second windows and converted into an 80-channel log-Mel spectrogram (25 ms window, 10 ms hop). The Transformer encoder turns the spectrogram into contextual audio embeddings; the decoder then generates text tokens autoregressively using cross-attention over those embeddings. Special tokens control the task (<|transcribe|> or <|translate|>), the language and timestamp prediction. Beam search keeps the `beam_size` most probable hypotheses at every step. When `vad_filter` is enabled, Silero VAD first removes non-speech regions, which reduces both runtime and hallucinated text on silence.

```
segments_raw, info = self.model.transcribe(
    audio_path, beam_size=cfg.beam_size, task=cfg.task.value,
    language=cfg.language, vad_filter=cfg.vad_filter,
    word_timestamps=cfg.word_timestamps,
    temperature=cfg.temperature, initial_prompt=cfg.initial_prompt)

for idx, seg in enumerate(segments_raw):
    confidence = round(float(np.exp(seg.avg_logprob)), 3)
    segments.append({"id": idx, "start": round(seg.start, 3),
                     "end": round(seg.end, 3), "text": seg.text.strip(),
                     "confidence": confidence, "words": words_list})
```

### 10.3 Subtitle Timecode Formatting

SRT and VTT differ only in the header and in the millisecond separator:

```
millis = int(round((seconds % 1) * 1000))
hrs, mins, secs = int(seconds)//3600, (int(seconds)//60) % 60, int(seconds) % 60
srt = f"{hrs:02d}:{mins:02d}:{secs:02d},{millis:03d}"   # 00:00:03,420
vtt = f"{hrs:02d}:{mins:02d}:{secs:02d}.{millis:03d}"   # 00:00:03.420
```

**Sample .srt output**

```
1
00:00:00,000 --> 00:00:03,420
Welcome to the Automatic Speech
Recognition Studio demonstration.
```

**Sample .vtt output**

```
WEBVTT

00:00:00.000 --> 00:00:03.420
Welcome to the Automatic Speech
Recognition Studio demonstration.
```

### 10.4 Synthetic Speech Signal

For benchmarking without real recordings, a 440 Hz carrier is amplitude-modulated at a 3 Hz “syllable rate” and mixed with light Gaussian noise:

$$s(t) = \sin(2\pi \cdot 440\,t) \cdot \tfrac{1}{2}\big(1 + \sin(2\pi \cdot 3\,t)\big) + \mathcal{N}(0,\, 0.02^2)$$

The signal is normalised and stored as 16-bit mono PCM at 16 kHz.

# 11. USER INTERFACE WALKTHROUGH

## 11.1 Studio Landing Page & Model Configuration

On opening `http://localhost:3000`, the user sees the hero banner with capability badges, the theme selector, and four navigation tabs. The default *Studio Audio Transcribe* tab shows the Model Configuration panel on the left (model tier, transcribe/translate toggle, source language, beam size slider, VAD and word-timestamp switches) and the audio source card on the right with *Upload File* and *Live Mic* modes.

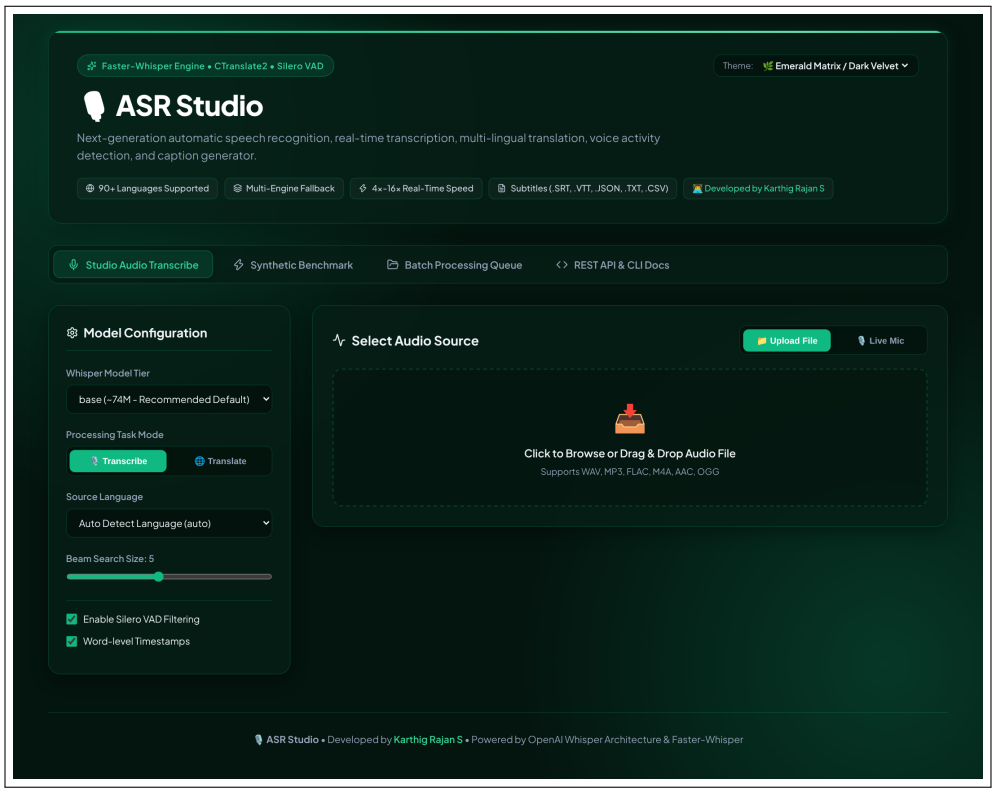



*Figure 4: ASR Studio main interface with Model Configuration panel and audio source selector (Emerald Matrix theme)*

## 11.2 Audio Upload, Live Recording & Signal Analysis

Dropping a file or finishing a recording immediately calls `POST /audio-info`; the returned duration, sample rate, RMS energy and peak amplitude are shown as metric cards together with an HTML5 audio player. During live recording, the canvas visualiser draws 64 frequency bars from the microphone stream at the browser's animation frame rate, and a timer shows the elapsed recording time.

## 11.3 Transcription Results & Export Hub

Clicking *Transcribe* sends the file and the selected settings to `POST /transcribe`. The results area displays the full transcript with a copy button, metadata cards (language and probability, duration, processing time, engine), the timed segment list with confidence scores, and five download buttons (TXT, SRT, VTT, JSON, CSV) which call `/transcribe/export`.

*Screenshot placeholder*
*(upload `figures/results.png`)*

*Figure 5: Transcription results with timed segments, confidence scores and export buttons*

### 11.4 Synthetic Speech Benchmark

The *Synthetic Benchmark* tab lets the evaluator choose a signal duration (1–10 s), generate the test WAV through `/synthesize-sample`, play it back, and run it through the full transcription pipeline. The measured processing time and real-time factor are displayed, allowing a quick check of performance on any machine without needing a recorded voice file.

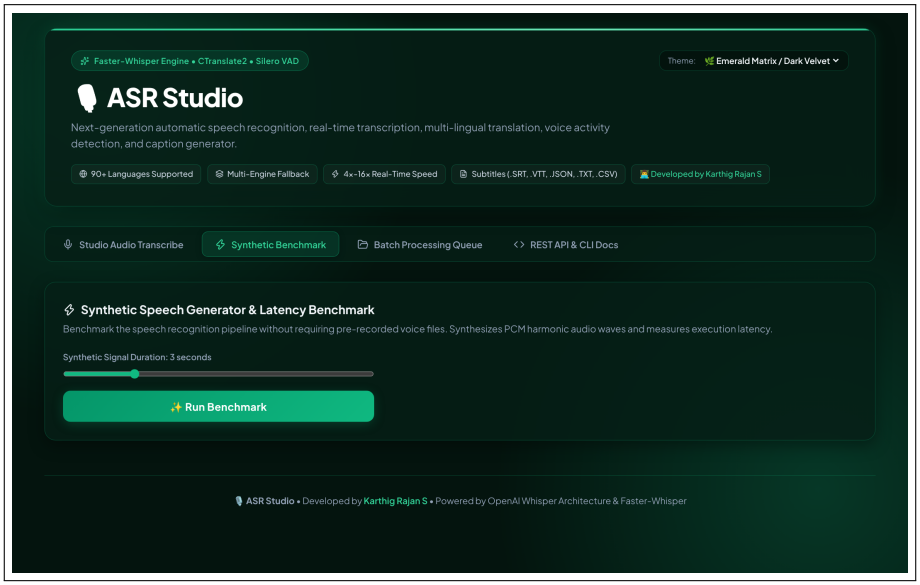



*Figure 6: Synthetic speech generator and latency benchmark tab*

### 11.5 Batch Processing Queue

The *Batch Processing Queue* tab accepts multiple files at once. They are transcribed one after another with the current model settings; a progress bar and per-file status cards (pending, processing, done, error) give live feedback, and each finished card shows the transcript preview.

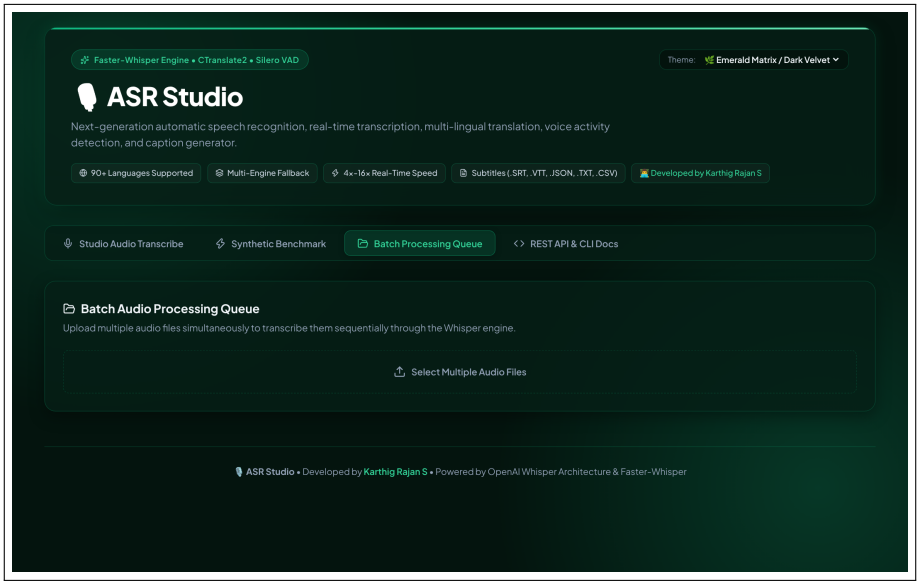



*Figure 7: Batch processing queue for multi-file transcription*

### 11.6 REST API & CLI Documentation Tab

The final tab documents every endpoint and CLI command with copy-ready examples, so developers can integrate ASR Studio into their own scripts or applications.

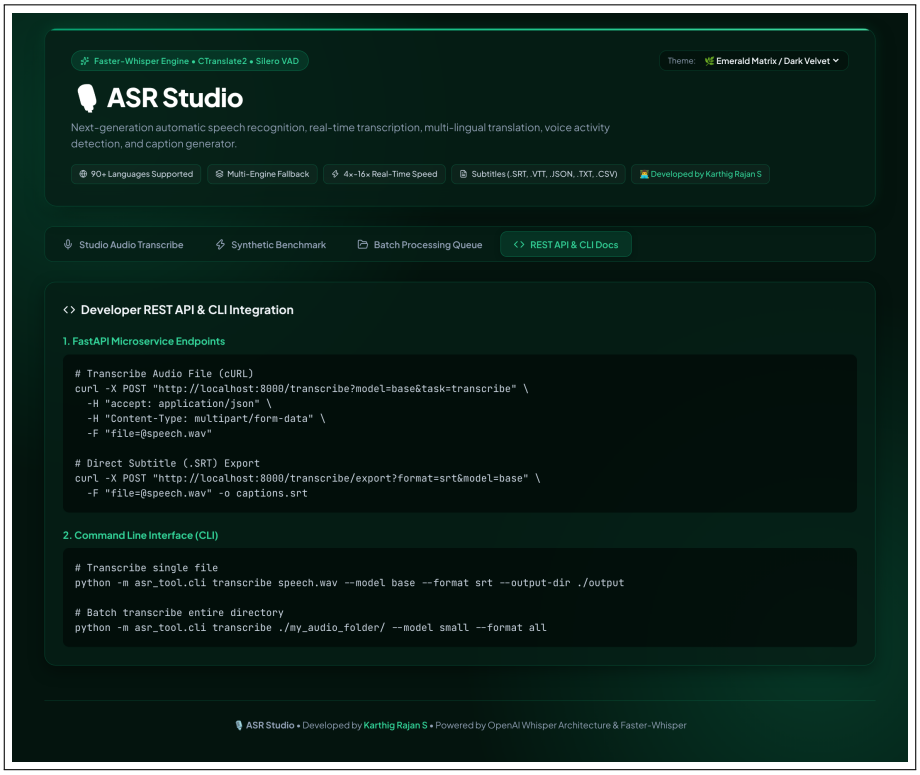



*Figure 8: In-app REST API and CLI reference*

### 11.7 Dynamic Theme Engine

Themes are implemented with CSS custom properties scoped by a `data-theme` attribute on the root `<html>` element. Selecting a theme in the dropdown updates this attribute through a React `useEffect` hook, and every component re-colours instantly without reloading.

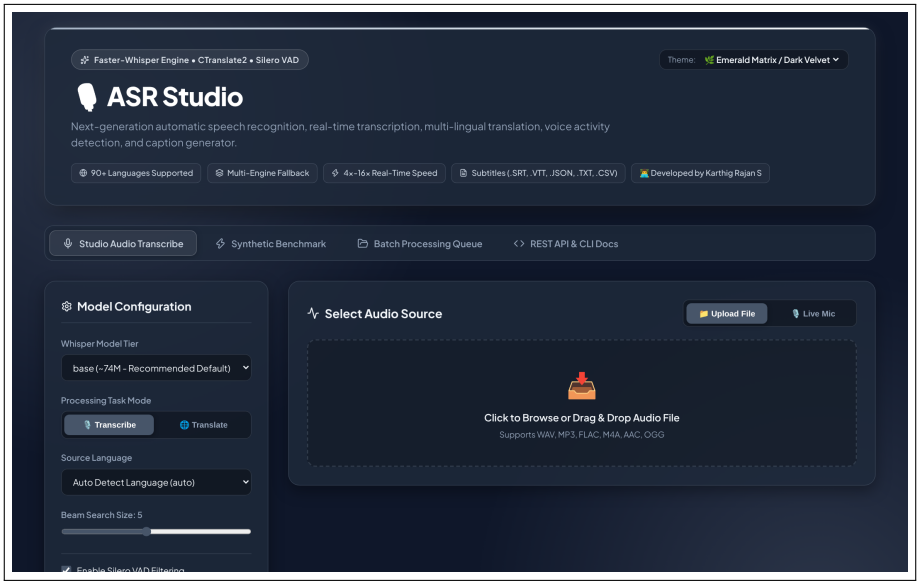



*Figure 9: The same studio rendered with the Nordic Frost theme*

| Theme | Visual Character |
|---|---|
| Emerald Matrix / Dark Velvet | Default; deep green background with emerald accents |
| Cyber Obsidian | Near-black surfaces with neon cyan highlights |
| Deep Nebula / Aurora | Purple-violet gradients |
| Tokyo Sunset / Horizon | Warm orange and pink tones |
| Nordic Frost / Minimal | Light, minimal palette for bright environments |

## 12. COMMAND-LINE INTERFACE

The same engine can be used without the browser through `python -m asr_tool.cli`:

| Sub-command | Purpose and Key Options |
|---|---|
| `transcribe` | File or directory (recursive) input; `-m` model, `-f` txt/srt/vtt/json/csv/all, `-l` language, `-t` transcribe/translate, `-o` output dir, `-device` cpu/cuda, `-beam-size` |
| `info` | Prints sample count, sample rate, duration, RMS, peak and silence flag |
| `generate-sample` | Writes a synthetic test WAV; `-d` duration in seconds |

```
# Transcribe one file and export every format
python -m asr_tool.cli transcribe lecture.mp3 -m small -f all -o output/

# Translate a whole folder of Hindi audio into English subtitles
python -m asr_tool.cli transcribe recordings/ -l hi -t translate -f srt

# Inspect an audio file / create a test signal
python -m asr_tool.cli info lecture.mp3
python -m asr_tool.cli generate-sample sample.wav -d 5
```

## 13. INSTALLATION & DEPLOYMENT GUIDE

**Prerequisites:** Python 3.10+, Node.js 18+ with npm, Git, and a modern browser (Chrome, Edge or Firefox) with microphone permission. FFmpeg is recommended for compressed formats.

```
# Step 1: Clone the repository
git clone https://github.com/Rajan3103/asr-studio.git
cd asr-studio

# Step 2: Create and activate a virtual environment
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate       # Linux / macOS

# Step 3: Install backend dependencies
pip install -r requirements.txt

# Step 4: Start the FastAPI backend (http://localhost:8000)
uvicorn asr_tool.api:app --host 0.0.0.0 --port 8000 --reload

# Step 5: In a second terminal, install and start the frontend
cd frontend
npm install
npm run dev                     # opens http://localhost:3000

# Step 6 (optional): run the test suite
pytest tests/ -v
```

On first use of a model tier, Faster-Whisper downloads the converted weights once and caches them locally; all later runs of that tier work offline.

## 14. TESTING & QUALITY ASSURANCE

ASR Studio includes 20 automated tests written with PyTest. API tests use FastAPI's `TestClient` (built on HTTPX), and audio tests generate their own synthetic WAV files in temporary directories, so the suite needs no external fixtures.

| Test Identifier | Target Component | Validation Scenario | Result |
|---|---|---|---|
| test_root_endpoint | API (GET /) | Status "online" and service metadata | PASSED |
| test_health_endpoint | API (GET /health) | Returns status "healthy" | PASSED |
| test_models_endpoint | API (GET /models) | Lists model tiers and languages | PASSED |
| test_transcribe_endpoint | API (POST /transcribe) | Synthetic WAV returns text, segments, engine | PASSED |
| test_transcribe_export_endpoint | API (/transcribe/export) | Returns downloadable subtitle file | PASSED |
| test_create_synthetic_speech_wav | AudioProcessor | WAV file created with correct format | PASSED |
| test_load_audio | AudioProcessor | Mono float32 output at 16 kHz | PASSED |
| test_get_audio_info | AudioProcessor | Duration, RMS and peak computed correctly | PASSED |
| test_save_wav | AudioProcessor | 16-bit PCM WAV written and readable | PASSED |
| test_cli_parser_build | CLI | Sub-commands and options parsed | PASSED |
| test_cli_generate_sample | CLI | generate-sample writes a WAV file | PASSED |
| test_cli_info | CLI | info prints audio metrics | PASSED |
| test_speech_recognition_engine_fallback | Fallback Engine | Fallback engine returns valid structure | PASSED |
| test_unified_asr_engine_transcribe | Engine | Unified engine returns full result schema | PASSED |
| test_to_txt | Exporter | Plain text output | PASSED |
| test_to_srt | Exporter | SRT numbering and , timecodes | PASSED |
| test_to_vtt | Exporter | WEBVTT header and . timecodes | PASSED |
| test_to_json | Exporter | Valid, round-trippable JSON | PASSED |
| test_to_csv | Exporter | Header row and segment rows | PASSED |
| test_export_to_file | Exporter | File written for the requested format | PASSED |

**Test execution command:** `pytest tests/ -v` (Result: 20 passed)

### 14.1 Manual & Integration Testing

- Uploaded each supported format (WAV, MP3, FLAC, M4A, AAC, OGG) and verified metrics and transcripts.
- Recorded live speech in Chrome and Edge and checked visualiser, timer and transcription.
- Opened generated .srt and .vtt files in VLC Media Player to confirm subtitle timing.
- Imported .csv exports into a spreadsheet to verify column layout.
- Tested translate mode with non-English speech and auto language detection.

- Verified that invalid parameters (e.g. `beam_size=20`) are rejected with HTTP 422.

## 15. PERFORMANCE METRICS & BENCHMARKS

Performance is reported using the *Real-Time Factor* (RTF), the ratio of processing time to audio duration:

$$\text{RTF} = \frac{T_{\text{processing}}}{T_{\text{audio}}}, \qquad \text{Speed-up} = \frac{1}{\text{RTF}}$$

An RTF below 1.0 means the audio is transcribed faster than real time.

| Metric Dimension | Observed Value | How It Was Measured |
|---|---|---|
| RTF – tiny (int8, CPU) | – | 60 s English speech clip, beam size 5 |
| RTF – base (int8, CPU) | – | Same clip, default settings |
| RTF – small (int8, CPU) | – | Same clip |
| First-request model load (base) | – | Time of first `/transcribe` call after server start |
| Cached request overhead | – | Repeated call minus inference time |
| Server RAM (base loaded) | – | Task Manager / `htop` |
| Test hardware | – | CPU model, cores, RAM |

### 15.1 Factors Affecting Performance

- **Model size:** larger tiers improve accuracy but increase latency and memory roughly in proportion to parameter count.
- **int8 quantisation:** stores weights in 8 bits, cutting memory use and speeding up CPU matrix multiplication with minimal accuracy loss.
- **Model caching:** the first request loads the model from disk; subsequent requests reuse the cached instance.
- **VAD filtering:** skipping silence reduces the amount of audio the decoder must process.
- **Beam size:** a smaller beam is faster; a larger beam can improve accuracy on difficult audio.

## 16. SECURITY & PRIVACY CONSIDERATIONS

- **On-Device Primary Inference:** with Faster-Whisper, audio is processed entirely on the host machine; no audio is sent to an external service.
- **Fallback Transparency:** the secondary SpeechRecognition engine uses the online Google Web Speech service. It runs only if Whisper fails, and the response then carries `fallback_triggered` and `fallback_reason` so the user knows. Privacy-critical deployments can disable this fallback.
- **Ephemeral Storage:** uploaded audio is saved to a temporary file only for the duration of the request and deleted in a `finally` block, even when processing fails.
- **Input Validation:** enum-typed and range-checked query parameters prevent invalid model names or beam sizes from reaching the engine.
- **Browser Sandboxing:** microphone capture requires explicit user permission through the browser.

- **Deployment Hardening:** CORS is currently open (`allow_origins=["*"]`) for local development; production deployments should restrict origins, add authentication, enforce upload size limits and serve over HTTPS.

## 17. COMPARISON WITH COMMERCIAL SOLUTIONS

Feature matrix comparing ASR Studio with widely used transcription services (pricing and features as publicly listed; they change over time):

| Dimension | ASR Studio | Otter.ai | Google Cloud STT | AWS Transcribe |
|---|---|---|---|---|
| Hosting | On-device | Cloud | Cloud | Cloud |
| Cost | Free, open source (MIT) | Subscription | Pay per minute | Pay per minute |
| Offline use | Yes (primary engine) | No | No | No |
| Speech translation to English | Built in | Limited | Separate API | Separate service |
| SRT / VTT export | Built in | Paid plans | Manual | Yes |
| Model choice | 5 tiers | Fixed | Several | Fixed |
| Self-hostable API | Yes | No | No | No |
| Speaker diarisation | Not yet | Yes | Yes | Yes |

## 18. LIMITATIONS

- Transcription happens after recording stops; there is no live word-by-word streaming yet.
- No speaker diarisation, so multi-speaker recordings are not labelled per speaker.
- Large models (medium, large-v3) are slow on CPU-only machines.
- The export endpoint re-runs transcription instead of reusing the previous result.
- No user accounts or transcript history; results are not stored on the server.

## 19. FUTURE ENHANCEMENTS

- **Real-Time Streaming:** WebSocket endpoint feeding audio chunks to the model for live captions.
- **Speaker Diarisation:** integrate pyannote.audio to label “Speaker 1”, “Speaker 2”.
- **Indian Language Focus:** fine-tuned models for Tamil, Hindi, Telugu and code-mixed speech.
- **Result Caching:** hash uploaded audio and reuse results for exports.
- **Transcript History:** optional SQLite/PostgreSQL storage with search.
- **Containerised Deployment:** Docker images with GPU support for lab servers.
- **Summarisation:** optional local LLM to generate meeting notes or lecture summaries.

## 20. CONCLUSION & REFERENCES

ASR Studio demonstrates that modern, open-source speech recognition can be packaged into a complete, user-friendly application that runs on everyday hardware. By combining Faster-Whisper's int8-quantised Transformer inference with a FastAPI microservice and a React studio interface, the project delivers accurate multilingual transcription, English translation, timed segments and five subtitle/export formats without depending on paid cloud APIs for its primary path. The modular design – separate preprocessing, engine, exporter, API and CLI layers – keeps the code testable, as shown by the 20 passing automated tests, and makes the engine reusable from a browser, a script or another service.

### Key Technical Milestones Achieved

- On-device Whisper inference with int8 CPU quantisation and per-tier model caching.
- Unified audio pipeline converting six formats and live recordings into 16 kHz mono signals.
- Graceful engine fallback with transparent reporting.
- TXT, SRT, VTT, JSON and CSV export from both the web studio and the CLI.
- React studio with live visualiser, benchmark, batch queue and five themes.

### References

1. Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision*. arXiv:2212.04356.
2. Vaswani, A., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS).
3. Klein, G., et al. (2020). *Efficient and High-Quality Neural Machine Translation with OpenNMT*. Proc. Workshop on Neural Generation and Translation. (CTranslate2 inference engine.)
4. SYSTRAN. *faster-whisper: Faster Whisper transcription with CTranslate2*. https://github.com/SYSTRAN/faster-whisper
5. Silero Team. *Silero VAD: pre-trained enterprise-grade Voice Activity Detector*. https://github.com/snakers4/silero-vad
6. Ramírez, S. *FastAPI Documentation*. https://fastapi.tiangolo.com
7. W3C. *Web Audio API* and *MediaStream Recording* specifications.
8. Meta Open Source. *React Documentation*. https://react.dev

### Project Links

**GitHub Repository:** https://github.com/Rajan3103/asr-studio
**Video Presentation:** https://drive.google.com/file/d/19zCYGW_cIZd9RbbAUSxV_bN3umef547Z/view?usp=sharing

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**
**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY, TIRUCHIRAPPALLI**